

dia 1: 11 de Diciembre

1. Diseño y Planificación: El Diagrama UML

Antes de escribir cualquier código, creamos un plano llamado **Diagrama UML**. Este dibujo es el mapa que nos dice cómo se van a llevar entre sí todas las clases del programa.

¿Cómo funciona este diagrama?

La Herencia): Usamos flechas para decir que una clase "viene de" otra. Por ejemplo, el Cajero y el Farmaceutico tienen una flecha hacia Empleado. Esto significa que ambos **son** empleados y comparten cosas como el nombre y el DNI, pero cada uno tiene su propia forma de ser.

Clases Abstractas: Dibujamos a Empleado y a Pago de forma especial. Son clases que no se pueden crear por sí solas (no puedes contratar a un "Empleado" a secas, tiene que ser un Cajero o un Farmacéutico). Sirven para obligar a las demás a tener métodos como calcularSalario() o procesarPago().

La Relación de Venta: La clase Venta aparece conectada a Medicamento y a Pago. Esto nos dice que para que exista una venta, el código debe buscar un producto y elegir una forma de cobrarlo.

2. Estructura en Spring Tool Suite (STS)

Una vez hecho el plano, abrimos la herramienta **Spring Tool Suite**. Esta herramienta nos ayuda a organizar el código de forma profesional para que no sea un desorden de archivos.

¿Qué hace cada carpeta (paquete)?

Dividimos el proyecto en tres capas principales. Así funciona la estructura:

A. Paquete de Modelos (com.Farmacia.Model)

Qué hace: Aquí guardamos los "moldes". Por ejemplo, el código de Medicamento.java.

Cómo funciona: Este código define qué datos tiene un producto (ID, nombre, precio). Es como la base de datos de los objetos. Si el programa fuera una cocina, los modelos serían los ingredientes.

B. Paquete de Servicios (com.Farmacia.Service)

Qué hace: Aquí está la inteligencia o "la lógica". Por ejemplo, el código que revisa si hay stock.

Cómo funciona: El servicio toma los modelos y hace cosas con ellos. Es el encargado de decir: "Si hay stock, resta 1 y guarda la venta". Siguiendo el ejemplo de la cocina, los servicios serían las recetas y los cocineros.

C. Paquete de Controladores (com.Farmacia.Controller)

Qué hace: Es la puerta de entrada para el usuario.

Cómo funciona: Aquí el código espera que alguien le pida algo desde internet (un "endpoint"). Cuando alguien pide ver los medicamentos, el controlador llama al servicio para que le dé la lista. En la cocina, el controlador sería el mozo que toma el pedido del cliente.

Día 2: 12 de Diciembre

En este segundo día de trabajo, me enfoqué en programar las piezas fundamentales del sistema: quiénes trabajan en la farmacia (Empleados), qué se vende (Medicamentos) y cómo se registra esa salida (Ventas).

1. Los Trabajadores: La clase Empleado y sus especialidades

Para que la farmacia funcione, necesitamos personas. Usando el plano (UML) del primer día, programé cómo el sistema entiende a los empleados.

Clase Empleado.java

¿Qué hace?: Es la base de todos los trabajadores. Guarda los datos que todos tienen en común: su nombre y su número de DNI.

¿Cómo funciona?: Es una clase "abstracta", lo que significa que no podemos crear un empleado "a secas". Su función principal es obligar a que cualquier tipo de empleado que agreguemos después tenga que explicar cómo cobra su sueldo a través del método calcularSalario().

Clase Farmaceutico.java

¿Qué hace?: Representa al profesional técnico de la farmacia.

¿Cómo funciona su código?: Es muy simple. Este empleado tiene un "sueldo fijo". Cuando el programa le pregunta cuánto cobra, el código simplemente responde con ese valor guardado, sin hacer cuentas extras.

Clase Cajero.java

¿Qué hace?: Representa al personal que cobra por el tiempo trabajado.

¿Cómo funciona su código?: A diferencia del farmacéutico, este código sí hace una cuenta matemática. Toma las horasTrabajadas y las multiplica por el valorHora. El resultado de esa multiplicación es el salario que el sistema registra para este empleado.

2. Lo que vendemos: La clase Medicamento

Sin productos no hay farmacia, por eso programé el corazón del inventario.

Clase Medicamento.java

¿Qué hace?: Es la ficha técnica de cada remedio. Guarda el nombre, la categoría (por ejemplo, "Analgesico"), el precio y cuántas cajas hay en el estante (stock).

¿Cómo funciona su código?: Tiene "sensores" llamados *getters* para que el sistema pueda leer el precio o el nombre en cualquier momento. También tiene un método muy importante llamado *setStock*. Este es el que permite que, cada vez que se hace una venta, el número de cajas disponibles baje automáticamente en el sistema.

3. La operación: La clase Venta

Este código es el que une a los productos con el dinero.

Clase Venta.java

¿Qué hace?: Registra el momento exacto en que algo sale de la farmacia.

¿Cómo funciona su código?: Cuando ocurre una venta, este código junta tres cosas: el Medicamento que se llevan, la cantidad y el objeto Pago.

La lógica del total: Lo más interesante es que la clase Venta no sabe cuánto cobrar por sí sola; en lugar de eso, le pregunta al objeto Pago: "Hacé tu trabajo y decime el total" mediante el método *calcularTotal()*. Esto permite que, si es efectivo, se aplique el descuento que programamos antes, o si es tarjeta, se aplique el interés. Además, guarda automáticamente la fecha del día para que sepamos cuándo se vendió cada cosa.

Día 3: 13 de Diciembre

En este tercer día, programé la inteligencia financiera del sistema y las "ventanillas de atención" que permiten que una persona use el programa desde su computadora.

1. El Manejo del Dinero: Clase Pago y sus variantes

Como en una farmacia no todos pagan de la misma forma, programé un sistema que entiende si el cliente usa billetes o plástico.

Clase Pago.java (La base)

¿**Qué hace?**: Es el molde para cualquier cobro. Guarda el monto inicial de la compra.

¿**Cómo funciona?**: Es abstracta, lo que significa que solo sirve para asegurar que todos los pagos tengan un método llamado procesarPago(). Es como decir: "No importa cómo pagues, el sistema tiene que saber calcular el precio final".

Clase PagoEfectivo.java (Subclase)

¿**Qué hace?**: Se encarga de las ventas al contado.

¿**Cómo funciona el código?**: Recibe el monto y un porcentaje de descuento. Al procesar el pago, el código hace una resta: $\text{monto} - (\text{monto} * \text{descuento})$.

Resultado: Si un remedio cuesta \$1000 y hay 10% de descuento, el código devuelve \$900.

Clase PagoTarjeta.java (Subclase)

¿**Qué hace?**: Se encarga de las ventas con crédito o débito.

¿**Cómo funciona el código?**: A diferencia del efectivo, aquí el código suma un interés. Además, guarda en cuántas cuotas se hace la compra.

Resultado: Si hay un 15% de interés, el código transforma esos \$1000 iniciales en \$1150.

2. Las Ventanillas del Sistema: Los Controllers

Los controladores son los que reciben las órdenes del usuario. Sin ellos, el programa funcionaría pero nadie podría tocarlo.

MedicamentoController.java

¿**Qué hace?**: Es el encargado de recibir pedidos sobre los remedios.

¿**Cómo funciona el código?**:

Usa @PostMapping para que el usuario pueda enviar un remedio nuevo y "crearlo" en el sistema.

Usa @GetMapping para mostrar la lista de todos los remedios o buscar uno solo por su ID.

Tiene una función especial llamada `stockCritico`. Cuando el usuario entra ahí, el código le muestra rápido qué remedios se están por acabar para que pida más al depósito.

VentaController.java

¿Qué hace?: Es el que maneja la caja registradora virtual.

¿Cómo funciona el código?:

Cuando alguien quiere comprar, el controlador recibe tres datos: el ID del remedio, la cantidad y cómo va a pagar (`tipoPago`).

Este código toma esos datos y se los pasa al "cerebro" (el `Service`) para que descuente el stock y haga la cuenta del dinero.

También tiene una función llamada `totalVendido`. Es como apretar un botón al final del día para saber cuánta plata entró en total a la farmacia.

Día 4: 14 de Diciembre

En este cuarto día, programé la lógica avanzada. Los **Services** son los que realmente mandan en el programa: ellos deciden si una venta es válida o si un medicamento está bien cargado. También hice los **Tests**, que son programas chiquitos que sirven para ver si el código principal tiene errores.

1. El Cerebro del Sistema: Los Services

MedicamentoService.java

¿Qué hace?: Controla todo el inventario de remedios.

¿Cómo funciona el código?:

Reglas de seguridad: Cuando agregamos un medicamento, el código revisa que el precio no sea 0 o negativo, y que tenga un nombre. Si el usuario se equivoca, el código lanza un "grito" (`Exception`) y no guarda nada.

Lista de faltantes: Tiene el método `stockCritico()`. Este código recorre todos los remedios y nos devuelve una lista solo de aquellos que tienen menos de 10 unidades.

Lógica: Funciona como un filtro para que los datos en el sistema siempre sean reales y correctos.

VentaService.java

¿**Qué hace?**: Es el encargado de realizar la venta paso a paso.

¿**Cómo funciona el código?**:

Paso 1 (Stock): Primero mira si hay suficientes cajas de remedios. Si pides 5 y hay 3, el código frena la venta.

Paso 2 (Dinero): Usa un switch para ver cómo paga el cliente. Si es "efectivo", el código llama automáticamente a la cuenta del descuento del 10%. Si es "tarjeta", le aplica el 15% de interés.

Paso 3 (Actualizar): Resta lo vendido del estante original para que el stock baje.

Lógica: Asegura que la farmacia nunca venda algo que no tiene y que cobre siempre el precio justo.

2. Control de Calidad: Los Tests

Los tests son como exámenes que le tomamos al código para ver si sabe hacer las cuentas.

MedicamentoTest.java y VentaTest.java

¿**Qué hacen?**: Verifican que los objetos se creen bien.

¿**Cómo funcionan?**: El código de prueba crea un remedio (por ejemplo, "Paracetamol") y luego usa una función llamada `assertEquals`. Esto es como decir: "Yo espero que el nombre sea Paracetamol, ¿es verdad?". Si es verdad, el test pasa. En la venta, verifica que después de vender, el stock haya bajado correctamente.

PagoEfectivoTest.java y PagoTarjetaTest.java

¿**Qué hacen?**: Verifican que la matemática de los descuentos e intereses no falle.

¿**Cómo funcionan?**:

El de **Efectivo** pone \$1000, aplica el 10% y espera ver \$900.

El de **Tarjeta** pone \$1000, aplica el 15% y espera ver \$1150.

Importancia: Esto nos da tranquilidad de que no le vamos a cobrar de más ni de menos a los clientes por un error de cuenta.

Día 5 y último día: 15 de Diciembre

El último día de trabajo se centró en dos partes fundamentales: asegurar que el "cerebro" del sistema no falle (Tests de Servicios) y poner en marcha la aplicación con datos reales para verla funcionar en la consola (Clase Main).

1. Pruebas de los Servicios (Service Tests)

Antes de dar el proyecto por terminado, probé las funciones más complejas para evitar errores humanos.

MedicamentoServiceTest.java

¿Qué hace?: Verifica que el sistema de inventario sea seguro.

Cómo funciona:

Prueba que al agregar un medicamento se le asigne un ID correctamente.

Intenta agregar un remedio con precio \$0\$ para confirmar que el sistema lo rechaza (lanza un error). Esto asegura que nadie cargue productos gratis por error.

VentaServiceTest.java

¿Qué hace?: Verifica que las ventas descuenten el stock y calculen bien el dinero.

Cómo funciona:

Simula una venta de 2 unidades y revisa si el estante bajó de 10 a 8.

Intenta vender 5 unidades cuando solo hay 1 disponible para verificar que el sistema bloquee la venta por "Stock insuficiente".

2. El Corazón del Programa: Main

Este es el archivo principal. Es el encargado de encender el motor de Spring Boot y ejecutar las primeras acciones del sistema apenas se abre.

¿Qué hace?: Usa una función llamada CommandLineRunner. Esto hace que, en cuanto el programa arranca, se ejecuten automáticamente unas órdenes en la pantalla negra (consola).

¿Cómo funciona el código?:

1. **Arranque:** El método main inicia Spring Boot.
2. **Carga de datos:** El código crea automáticamente dos medicamentos ("Paracetamol" e "Ibuprofeno") y los guarda en la lista.
3. **Simulación de ventas:** El programa realiza dos ventas solo: una en **efectivo** (donde aplica el descuento del 10%) y otra con **tarjeta** en 3 cuotas (donde aplica el interés del 15%).
4. **Cierre de caja:** Al final, imprime en la consola el "TOTAL VENDIDO", que es la suma de todas las operaciones realizadas.

3. Funcionamiento de la Arquitectura Final

Con la clase Main integrada, el flujo del sistema se ve así:

1. **Inicio:** FarmaciaApplication despierta a los servicios.
2. **Acción:** El usuario pide una venta a través de los **Controllers**.
3. **Procesamiento:** Los **Services** validan el stock y calculan el precio según el tipo de **Pago** (clases abstractas).
4. **Resultado:** Se actualiza el **Medicamento** (modelo) y se muestra el total en consola o pantalla.

Endpoints:

Estos endpoints están en el archivo MedicamentoController.java y sirven para manejar todo lo que hay en los estantes.

POST /medicamentos

¿Qué hace?: Registra un remedio nuevo en la lista.

Cómo funciona: El usuario envía los datos (nombre, precio, stock) y el controlador se los pasa al MedicamentoService. El servicio revisa que el precio sea mayor a cero y, si está todo bien, le asigna un número de ID y lo guarda.

GET /medicamentos

¿Qué hace?: Muestra una lista de todos los medicamentos que tenemos.

Cómo funciona: Llama al servicio, pide la lista completa que está en la memoria y la devuelve para que el usuario pueda verla.

GET /medicamentos/{id}

¿Qué hace?: Busca un solo remedio usando su número de identificación (ID).

Cómo funciona: El usuario pone el número en la dirección (ej: /medicamentos/5). El código busca en la lista y, si lo encuentra, lo muestra. Si no existe, lanza un error de "No encontrado".

DELETE /medicamentos/{id}

¿Qué hace?: Borra un medicamento del sistema.

Cómo funciona: Busca el remedio por su número y lo elimina de la lista definitiva. Es útil para productos que ya no se fabrican o se cargaron por error.

GET /medicamentos/critico

¿Qué hace?: Avisa qué remedios se están por acabar.

Cómo funciona: El sistema recorre todos los productos y filtra los que tienen **menos de 10 unidades** de stock. Devuelve una lista corta solo con esos productos.

2. Gestión de Ventas

Estos endpoints están en el archivo VentaController.java y sirven para manejar el dinero y las transacciones.

POST /ventas

¿Qué hace?: Realiza una venta oficial.

Cómo funciona: Recibe el ID del producto, la cantidad y el método de pago ("efectivo" o "tarjeta").

El controlador llama al VentaService.

El servicio resta las unidades del stock.

Aplica el descuento del 10% si es efectivo o el 15% de interés si es tarjeta.

Crea un registro con la fecha actual.

GET /ventas/total

¿Qué hace?: Dice cuánta plata ganó la farmacia en total.

Cómo funciona: Suma todos los montos de todas las ventas que se hicieron desde que se prendió el programa y devuelve el número final (ej: \$55.400).